MIPS Design and Simulation

CSE 210 Assignment 3

Lab Group: A1

Group: 8

Members:

2105022: Mafruha Islam Aishi
2105023: Shahraz Hussain Siyam
2105024: Ridhwanul Islam

1 Introduction

The MIPS (Microprocessor without Interlocked Pipeline Stages) architecture is a widely used RISC (Reduced Instruction Set Computing) architecture. One of the key components in a MIPS processor is the **datapath**, which is responsible for the flow of data through the processor and performs the necessary operations for executing instructions. The datapath consists of various components such as registers, ALU (Arithmetic Logic Unit), multiplexers, and memory units, all connected in a way that supports instruction execution.

In the MIPS datapath, data flows through different stages depending on the type of instruction being executed, whether it is a R-type, I-type, or J-type instruction. These instructions perform operations like arithmetic, logic, memory access, and control, which the datapath handles in conjunction with the control unit.

The main components of the MIPS datapath are:

- **Registers:** Hold temporary data, including the program counter (PC), general-purpose registers, and special registers.
- **ALU:** Performs arithmetic and logical operations.
- **Memory:** Includes both data memory and instruction memory for reading and writing values.
- **Multiplexers:** Select between different inputs to send to the appropriate component.
- **Control Unit:** Generates control signals that coordinate the operation of the datapath components.

The MIPS datapath allows for efficient and pipelined execution of instructions. Each instruction goes through stages such as fetching, decoding, executing, memory accessing, and writing back results. The efficient design of the datapath is crucial for achieving high-performance execution in MIPS processors.

2 Instruction Set

Table 1 below summarizes the MIPS instruction set, categorizing each instruction type (Arithmetic, Logic, Memory, Control) and providing the corresponding instruction.

Instruction ID	Instruction Type	Instruction
A	Arithmetic	add
B	Arithmetic	addi
C	Arithmetic	sub
D	Arithmetic	subi
E	Logic	and
F	Logic	andi
G	Logic	or
H	Logic	ori
I	Logic	sll
J	Logic	srl
K	Logic	nor
L	Memory	sw
M	Memory	lw
N	Control	beq
O	Control	bneq
P	Control	j

MIPS instruction set will be 20 bit in length with the following three formats

The following table shows the MIPS instruction names and their corresponding opcodes (in hexadecimal) for each character in the sequence ‘KMFJHLCBIGPEADNO‘.

Instruction-Type	Format				
R-type	Opcode 4-bits	Src Reg 1 4 bits	Src Reg 2 4 bits	Dst Reg 4 bits	Shft Amnt 4 bits
I-type	Opcode 4-bits	Src Reg 1 4 bits	Src Reg 2 4 bits	Address/Immediate 8 bits	
J-type	Opcode 4 bits	Target Jump Address 8 bits		0 4 bits	0 4 bits

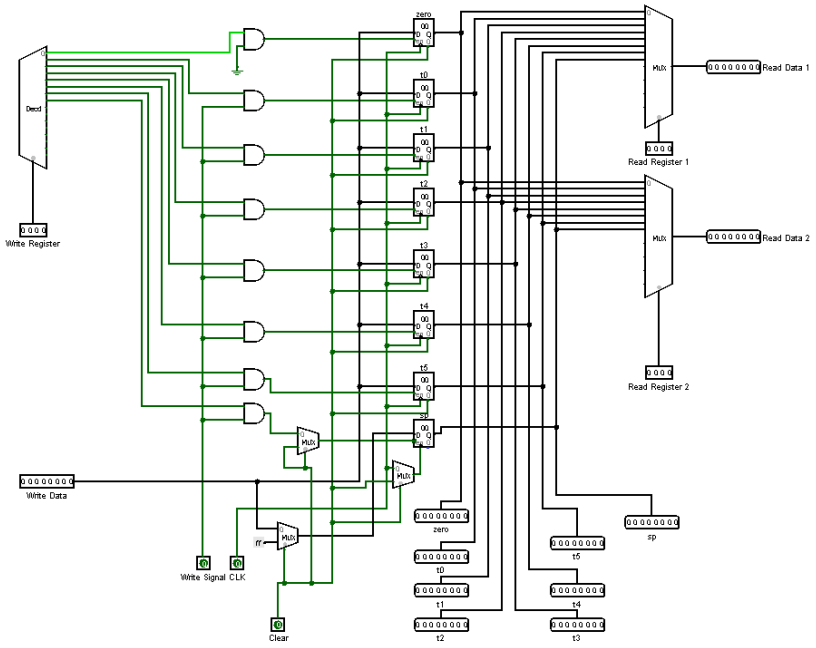
Character	**MIPS Instruction**	**Opcode (Hex)**
K	nor	0
M	lw	1
F	andi	2
J	srl	3
H	ori	4
L	sw	5
C	sub	6
B	addi	7
I	sll	8
G	or	9
P	j	A
E	and	B
A	add	C
D	subi	D
N	beq	E
O	bneq	F

3 Complete Circuit Diagram (of all components)

This section presents the complete circuit diagram for the MIPS processor, including all relevant components such as the Register File, ALU, Instruction Memory, Data Memory, and Control.

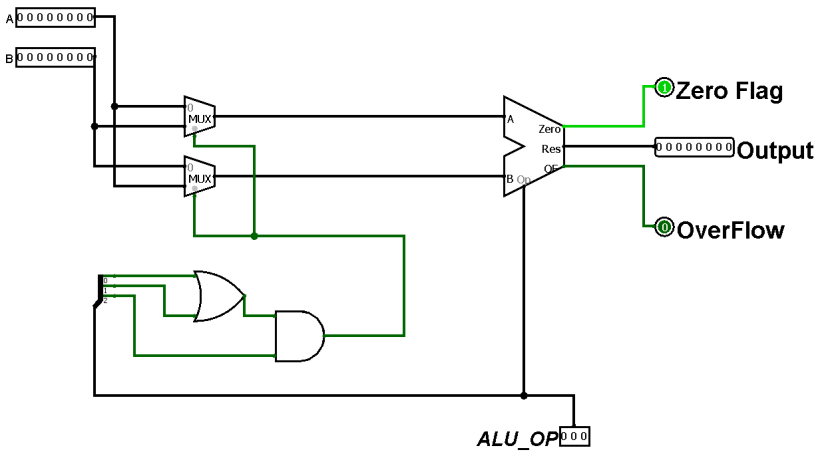
3.1 Register File

Below is the circuit diagram for the Register File, which contains the general-purpose registers used to hold data for operations:



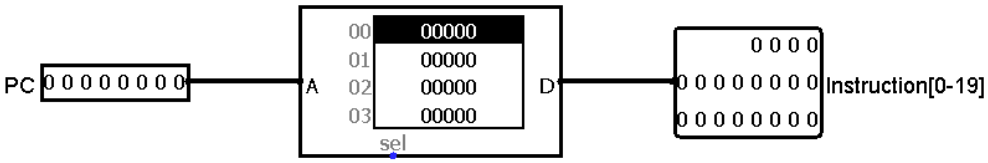
3.2 ALU (Arithmetic Logic Unit)

The ALU performs all arithmetic and logical operations, including addition, subtraction, AND, OR, and comparison operations:



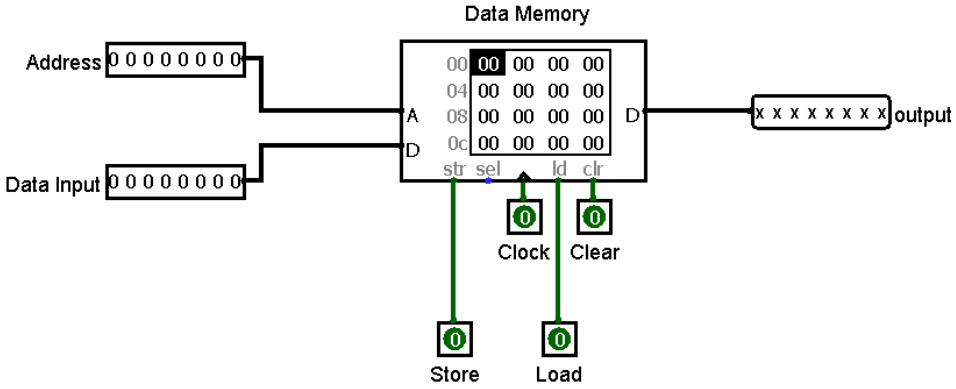
3.3 Instruction Memory

This component stores the instructions fetched during the program execution:



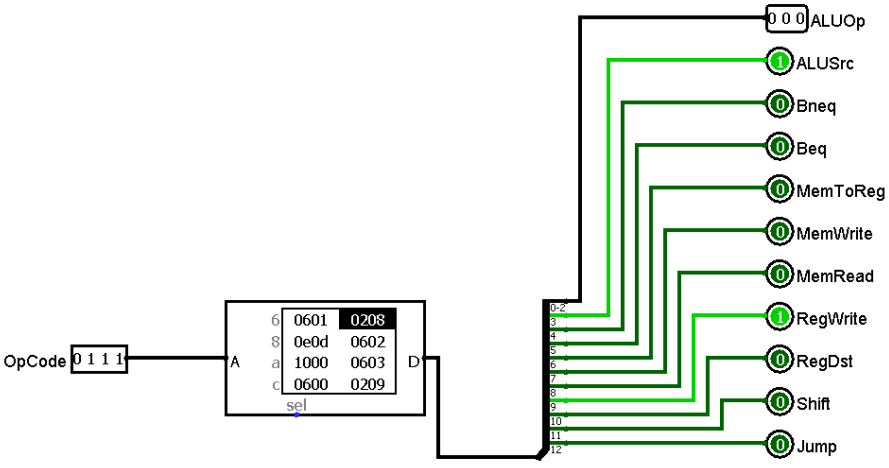
3.4 Data Memory

Data Memory is responsible for storing data that can be accessed or modified by the program:



3.5 Control

The Control unit generates the necessary control signals to orchestrate the operation of other components:



4 How to Write and Execute a Program on Your Machine

This section describes the process of writing and executing a program on the MIPS processor design.

Step 1: Writing the MIPS Program

The first step in executing a program is to write the MIPS assembly code. The code can be written using any standard text editor or Integrated Development Environment (IDE). The MIPS assembly language consists of instructions that the processor can decode and execute, such as load, store, arithmetic, and control operations.

Step 2: Assembling the MIPS Code

After the MIPS program has been written, it is passed through an assembler. The assembler converts the human-readable MIPS assembly code into machine-readable hexadecimal instructions. These instructions represent the binary encoding of the assembly instructions that the processor can understand. The resulting output is a series of hexadecimal values corresponding to the MIPS instructions.

Step 3: Loading the Instruction ROM

Once the hexadecimal machine code has been generated, it is loaded into the Instruction ROM. The Instruction ROM stores the instructions that the processor will fetch and execute. The processor fetches instructions from the Instruction ROM one at a time during the execution of the program.

Step 4: Loading the Control Signals from the Control File

In parallel with the instruction ROM, a separate control file is used to store the necessary control signals that dictate the operations of various components within the processor, such as the ALU, Register File, and Data Memory. These control signals are typically stored in a text file. The control file is loaded into the processor, allowing the control unit to generate the appropriate control signals for each instruction type.

Step 5: Executing the Program with Clock Pulses

The program is executed step by step based on clock pulses. Each clock pulse allows the processor to fetch the next instruction from the Instruction ROM. After fetching an instruction, the processor decodes it and generates the corresponding control signals based on the control file. These signals determine how the processor will execute the instruction, such as whether it needs to perform an arithmetic operation using the ALU or access data from the Data Memory.

The processor then writes the results of the instruction execution back to the registers or memory as needed. This cycle continues with each clock pulse, and the program progresses through its instructions in sequence.

5 ICs used with count as chart

Table 3: ICs used with count as a chart

Gate	Gate Count	IC	IC Count
MUX(4-to-1)	1	74153	1
MUX(2-to-1)	9	74157	3
MUX(8-to-1)	1	74151	1
MUX(16-to-1)	2	74150	2
Decoder(4-to-16)	1	74154	1
AND(2-bit)	10	7408	3
AND(8-bit)	1	7430	1
NOT(1-bit)	1	7404	1
NOR(8-bit)	1	4078	1
Priority Encoder(2-bit)	1	Priority Encoder(2-bit)	1
Register(8-bit)	9	Register(8-bit)	9
ROM(256X20)	1	ROM(256X20)	1
ROM(256X8)	1	ROM(256X8)	1
ROM(16X13)	1	ROM(16X13)	1
RAM(256X8)	1	RAM(256X8)	1
Clock	1	Clock	1
Right-Shifter(8-bit)	1	Right-Shifter(8-bit)	1
Left-Shifter(8-bit)	1	Left-Shifter(8-bit)	1

6 Contributions

- **Report**: 2105022, 2105023, 2105024
- **Software**: 2105022, 2105023, 2105024
- **Simulation**: 2105022, 2105023, 2105024

7 Discussion

Our implementation of the MIPS architecture was distinct in its simplicity and focus on foundational principles, relying entirely on software without incorporating any microcontroller. This approach not only streamlined our implementation but also posed its own unique set of challenges, particularly in accurately simulating the MIPS datapath and ensuring a smooth translation from assembly instructions to machine code.

One of the key features of our project was the development of a C++ program to convert assembly code into our corresponding 20-bit machine code. This tool automated the translation process, eliminating manual conversion errors and allowing us to test a wide variety of instructions efficiently. The program ensured precise alignment with the instruction formats we defined for R-type, I-type, and J-type instructions. While we did not implement pipelining in our architecture, our use of a single-cycle datapath simplified execution and made debugging more manageable. This decision aligned with our goal of prioritizing clarity and understanding over performance optimizations.

In terms of memory management, we took a unique approach by integrating the stack memory directly into our data memory. This eliminated the need for a separate stack module and reduced the overall complexity of our design. The unified memory architecture allowed us to implement stack operations seamlessly, leveraging the data memory for both variable storage and function calls, which was particularly effective for smaller-scale projects like ours.

To ensure modularity and maintainability, we implemented each component of the MIPS architecture—such as the instruction memory, control unit, ALU, data memory, and register file—as independent modules. This modular approach not only improved readability and ease of debugging but also allowed

us to focus on the individual functionalities of each component. For the arithmetic and logical operations, we utilized the ALU provided by our instructor, which ensured reliable and consistent results while saving us the effort of designing one from scratch.

Our implementation featured a straightforward execution process that adhered to the basic principles of the instruction cycle. Each instruction was fetched, decoded, and executed within a single clock cycle. While this limited our performance compared to pipelined architectures, it allowed us to focus on understanding the fundamentals of how each instruction traverses the datapath.

We dedicated significant effort to testing our implementation, running various test cases to validate the correctness of instruction execution and ensuring that all components interacted as expected. Debugging the software and verifying outputs against expected results were crucial steps in refining our implementation and gaining deeper insights into the MIPS architecture.

Overall, this project provided us with an excellent opportunity to explore the design and operation of a MIPS processor. By focusing on a single-cycle datapath and integrating stack memory into data memory, we achieved a simplified yet functional representation of MIPS. The modularity of our design and the automation of assembly-to-machine code conversion were highlights of our implementation, helping us achieve a solid understanding of MIPS architecture and the principles of instruction execution.